

Relatório Técnico e Acadêmico: Frameworks e Bibliotecas Frontend

Estudo Comparativo entre React, Angular, Vue.js e Svelte no Desenvolvimento Web Moderno

INTEGRANTES DO GRUPO

- Edilaine Paulino Soldé
- Fabio Bitencourt Ribeiro
- Gabriel Camargo Gonçalves Silva
- Guilherme Lima Leite
- Ronaldo Francisco Soares

Sorocaba, SP — 2026

1. Introdução e Contextualização Histórica

O desenvolvimento de software para a Web passou por profundas transformações ao longo das últimas três décadas. Inicialmente concebida como um sistema de documentos hipertexto estáticos, a Web evoluiu para se tornar a principal plataforma de entrega de aplicações complexas e interativas em tempo real.

Nos primórdios, as páginas web eram processadas quase exclusivamente no lado do servidor (*Server-Side Rendering - SSR*). A cada ação do usuário, como o envio de um formulário ou a navegação entre seções, o navegador solicitava um novo documento HTML completo. Esse modelo gerava recarregamentos constantes de página, alto consumo de largura de banda e uma experiência de usuário truncada.

A introdução do JavaScript no final dos anos 1990 e a consolidação das requisições assíncronas via AJAX nos anos 2000 permitiram que partes específicas da página fossem atualizadas sem a necessidade de recarregar todo o documento. No entanto, a manipulação direta do DOM (*Document Object Model*) por meio de rotinas imperativas Vanilla JS ou bibliotecas como o jQuery tornou-se insustentável para grandes projetos.

O surgimento das arquiteturas de **Single Page Application (SPA)** resolveu esses gargalos ao transferir a lógica de renderização e o gerenciamento de estado para o navegador do cliente. Para suportar esse novo paradigma de engenharia, nasceram as bibliotecas e frameworks modernos de frontend.

1.1 Desafios da Engenharia Frontend Moderna

Construir interfaces de usuário modernas envolve resolver problemas complexos que vão além do design visual. Dentre os principais desafios enfrentados pelas equipes de desenvolvimento, destacam-se:

- **Gerenciamento de Estado Concorrente:** Manter os dados da aplicação sincronizados entre múltiplos componentes de interface sem gerar inconsistências ou bugs de sincronização.
- **Desempenho de Renderização:** Minimizar operações custosas de alteração estrutural no DOM nativo, que causam problemas de desempenho conhecidos como *layout thrashing* e queda na taxa de quadros (FPS).
- **Escalabilidade e Manutenibilidade:** Garantir que bases de código contendo centenas de milhares de linhas possam ser expandidas por múltiplos times de desenvolvimento sem criar acoplamento indesejado.

2. Paradigmas Arquiteturais e Fundamentos

2.1 Programação Declarativa vs. Imperativa

A diferença fundamental entre a web tradicional e as ferramentas modernas reside no paradigma de programação adotado para gerenciar a interface.

No modelo imperativo, o desenvolvedor precisa detalhar explicitamente a sequência de instruções necessária para alterar o estado do DOM:

```
// Abordagem Imperativa (JS Nativo): const statusElem = document.getElementById('status');
statusElem.innerText = 'Carregando...'; statusElem.classList.add('active');
```

Já no paradigma declarativo, a interface é expressa como uma função direta do estado atual da aplicação ($UI = f(State)$). Quando o estado muda, o framework encarrega-se de atualizar a interface automaticamente.

2.2 O Funcionamento do Virtual DOM

Para viabilizar a programação declarativa com alto desempenho, ferramentas como React e Vue utilizam o conceito de **Virtual DOM**. Trata-se de uma árvore de objetos JavaScript leves armazenados na memória RAM que simulam a estrutura real do DOM do navegador.

Sempre que ocorre uma alteração no estado da aplicação, o fluxo de execução segue os seguintes passos rigorosos:

1. Uma nova árvore do Virtual DOM é gerada em memória refletindo o novo estado.
2. Um algoritmo de diferenciação (*Diffing Algorithm*) compara a nova árvore com a árvore anterior.
3. São identificadas com precisão apenas as ramificações que sofreram alterações reais.
4. Um lote mínimo de operações de escrita (*reconciliation*) é enviado para execução no DOM real do navegador, preservando a fluidez da interface.

3. Análise Individual das Tecnologias

3.1 React (Meta Platforms)

Lançado em 2013, o React não é classificado como um framework completo, mas sim como uma biblioteca focada estritamente na construção de interfaces baseadas em componentes. Ele popularizou a sintaxe JSX (JavaScript XML), permitindo mesclar marcação estrutural e lógica de programação no mesmo arquivo.

Com a introdução dos *React Hooks* em 2019, o ecossistema abandonou o uso de componentes baseados em classes orientadas a objetos, adotando uma abordagem puramente funcional. O React destaca-se por possuir a maior comunidade mundial e o maior número de vagas no mercado de tecnologia, embora delegue decisões arquiteturais como roteamento e gerenciamento de estado para bibliotecas de terceiros (Redux, Zustand, React Router).

3.2 Angular (Google)

O Angular é uma plataforma e framework "full-featured" que oferece uma solução ponta a ponta pronta para uso corporativo. Lançado em 2016 como sucessor do AngularJS, o framework foi desenvolvido do zero utilizando TypeScript nativamente.

Diferente de soluções focadas apenas na visão, o Angular fornece um conjunto integrado de ferramentas, incluindo cliente HTTP, sistema de roteamento com suporte a carregamento preguiçoso (*lazy loading*), injeção de dependências e validação de formulários reativos. Essa característica garante extrema padronização, sendo a escolha ideal para grandes empresas e equipes heterogêneas.

3.3 Vue.js (Evan You)

Criado por Evan You e mantido de forma independente por uma comunidade global, o Vue.js define-se como um framework progressivo. Isso significa que ele pode ser adotado gradualmente em projetos existentes como uma simples biblioteca de reatividade ou escalar para um framework completo com ecossistema oficial (Vue Router, Pinia).

O Vue utiliza o padrão de Arquivos de Componente Único (*Single File Components* - *.vue*), organizando template, lógica e estilo CSS em blocos coesos. Ele se destaca pela documentação exemplar, facilidade de aprendizado para iniciantes e excelente desempenho garantido por um sistema de reatividade baseado em Proxies do JavaScript.

3.4 Svelte (Rich Harris)

O Svelte propõe uma revolução técnica ao eliminar o conceito de Virtual DOM. Em vez de rodar um motor de execução pesado dentro do navegador do usuário, o Svelte atua como um **compilador em tempo de build**.

Durante o processo de compilação, o Svelte converte os componentes em código JavaScript Vanilla imperativo de altíssima eficiência. O resultado final enviado ao navegador possui um tamanho de pacote (*bundle size*) extremamente reduzido e consumo ínfimo de memória RAM, sendo altamente indicado para dispositivos com limitações de hardware.

4. Matriz Comparativa Detalhada

Característica	React	Angular	Vue.js	Svelte
Categoria	Biblioteca UI	Framework Completo	Framework Progressivo	Compilador JS
Linguagem Principal	JavaScript / JSX	TypeScript Nativo	JavaScript / HTML	JavaScript / HTML
Modelo de Renderização	Virtual DOM	Incremental DOM	Virtual DOM / Proxies	Compilação Direta no DOM
Curva de Aprendizado	Média	Alta	Baixa / Média	Baixa
Gerenciamento de Estado	Bibliotecas Externas	Injeção de Dependência / RxJS	Oficial (Pinia)	Stores Nativas
Tamanho do Bundle	Médio (~40 KB)	Grande (~150 KB+)	Pequeno (~30 KB)	Mínimo (<10 KB)

5. Recomendações de Adoção em Projetos Reais

A escolha de uma tecnologia frontend não deve se pautar por preferências individuais ou por engajamento em redes sociais. A decisão correta deriva da análise objetiva das necessidades do negócio, restrições orçamentárias e perfil técnico da equipe.

5.1 Diretrizes Práticas de Decisão

- **Projetos Enterprise / Corporativos de Grande Porte:** Recomenda-se a adoção do **Angular**. Sua estrutura opinativa impede que equipes numerosas criem padrões divergentes de código, garantindo manutenibilidade a longo prazo.
- **Aplicações Web voltadas ao Consumidor Final (SaaS, E-commerce):** Recomenda-se o **React** (frequentemente combinado com frameworks como Next.js), devido à imensa disponibilidade de bibliotecas de terceiros e facilidade de contratação de profissionais no mercado.
- **Prototipagem Rápida e Sistemas de Médio Porte:** O **Vue.js** sobressai-se pela rapidez no desenvolvimento e pela facilidade com que novos desenvolvedores absorvem a tecnologia.
- **Aplicações para Dispositivos com Restrição de Hardware (IoT, TVs, Mobile Leve):** O **Svelte** é a escolha ideal devido à ausência de sobrecarga de runtime e performance bruta imbatível.

6. Conclusão

As tecnologias de desenvolvimento frontend atingiram um patamar elevado de qualidade e robustez. Nenhuma das ferramentas analisadas (React, Angular, Vue.js ou Svelte) é universalmente superior às demais; cada uma oferece um conjunto diferente de vantagens e concessões (*trade-offs*).

A Engenharia de Software moderna exige que desenvolvedores e arquitetos compreendam os fundamentos subjacentes a essas ferramentas para realizar seleções conscientes que atendam às metas estratégicas dos projetos e das organizações.